



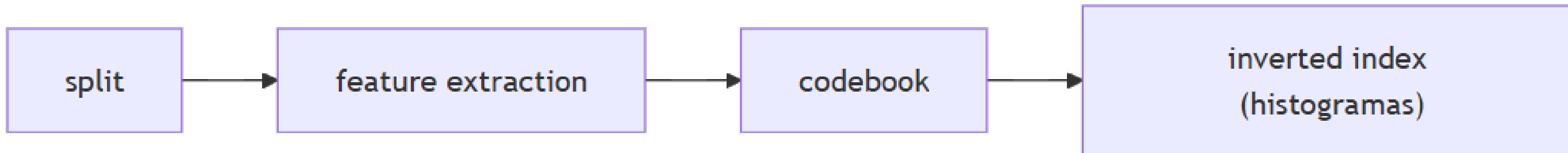
PROYECTO 2

Grupo 7 – UTEC



Introducción

MOTOR DE BASE DE DATOS MULTIMODAL



REQUISITOS

Funcionales

- **DDL:** CREATE TABLE
- **DML:** INSERTy DELETE con WHERE.
- **SELECT**
- **Modalidades:** **texto** (inverted index + TF-IDF), **imagen** (SIFT) y **audio** (MFCC por ventana o audio features del CSV)
- Planner que elige el índice según el predicado, y explain del plan
- **API y UI**
- **Comparación** contra índices nativos PostgreSQ (GIN y pgvector: IVFFlat y HNSW)

Predicado	Forma	Lo resuelve
Comparación	<code>id = 5</code> (también <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>)	hash extendible, o el índice ya creado sobre la columna
Rango	<code>id BETWEEN 1 AND 9</code>	B+Tree o ISAM
KNN por vector	<code>KNN(feats, [0.1, 0.2, 0.3], 5)</code>	MultimediaKNN
KNN por archivo	<code>KNN(img, "query.png", 5)</code>	MultimediaKNN, previa extracción y cuantización del archivo
Espacial	<code>WITHIN(box, [0, 0], [10, 10])</code>	R-Tree
Texto	<code>MATCH(lyrics, "términos", 3)</code>	inverted index (SPIMI + TF-IDF)
Híbrido	<code>HYBRID(cover, "query.png", lyrics, "términos", 3)</code>	KNN + inverted fusionados por ranking

REQUISITOS

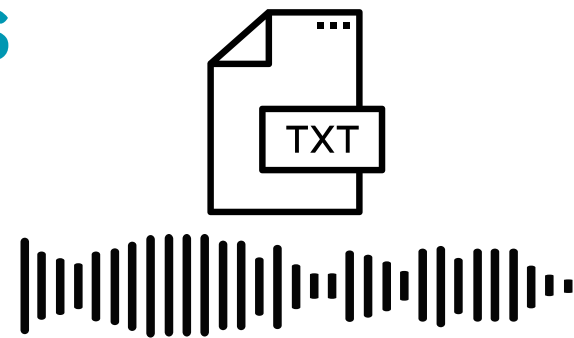
NO Funcionales

- **RAM acotada:** el buffer LRU retiene como máximo capacity páginas
- **Latencia** (medida en la corrida real de benchmarks): el texto propio pasa de 0.24 ms por consulta (N=1000) a 34.9 ms (N=100000) y el KNN propio de 0.14 ms a 47.0 ms. En la misma carga de 100K, IVFFlat responde en 2.7 ms y HNSW en 4.1 ms. La corrida de 100K chunks está registrada con sus tablas y plots en la sección de experimentos.
- **Portabilidad**
- **Observabilidad**
- **Testabilidad**

DATA-DATASETS

Texto y audio features

Canciones y letras entregadas por profesor



Audio real (opcional, FMA small)

8000 clips mp3 de 30 segundos del dataset público FMA

Imágenes (covers)

carpeta pública de Google Drive del proyecto



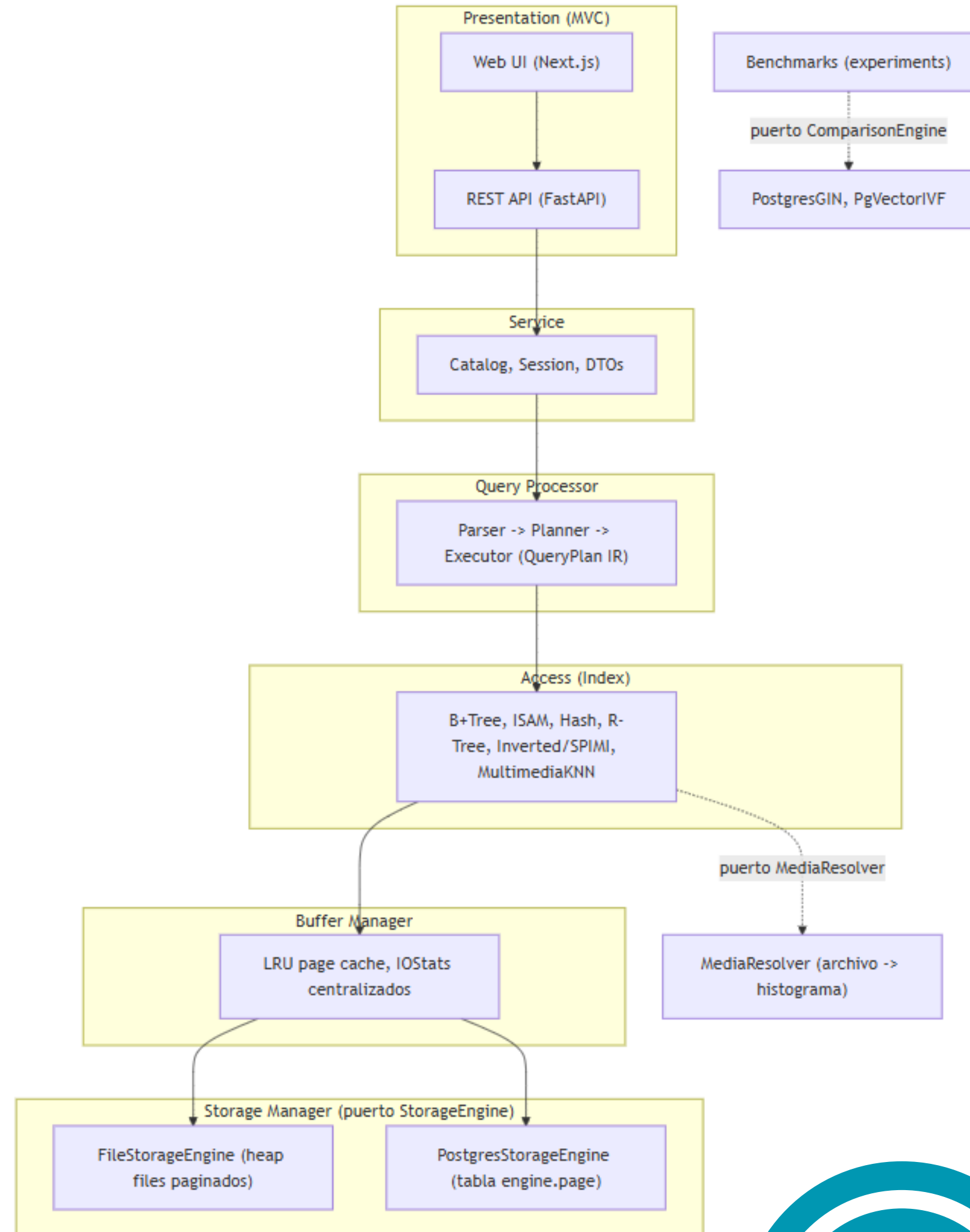
Sintético

cuando no hay data real, el seed genera un PNG y un WAV mínimos y los benchmarks generan documentos y vectores deterministas con seed fija.

DATA- ESTADÍSTICAS

Estadística	Valor
Canciones (mismos <code>track_id</code> en letras y features)	18194
Artistas únicos	5946
Palabras por letra: mediana / media	341 / 426.2
Palabras por letra: p25 / p75 / p95	235 / 510 / 991
Palabras por letra: mínimo / máximo	1 / 5509
Corpus total de letras	~7.75 millones de palabras (39 MB)
Audio features	12 columnas estandarizadas (media ~0, desviación ~1)
Covers JPG	17706 (960 MB)
Canciones con cover	17706 de 18194 (97.3 %)
Archivos de audio crudo	0 descargados por defecto (FMA small opcional: 8000)

ARQUITECTURA

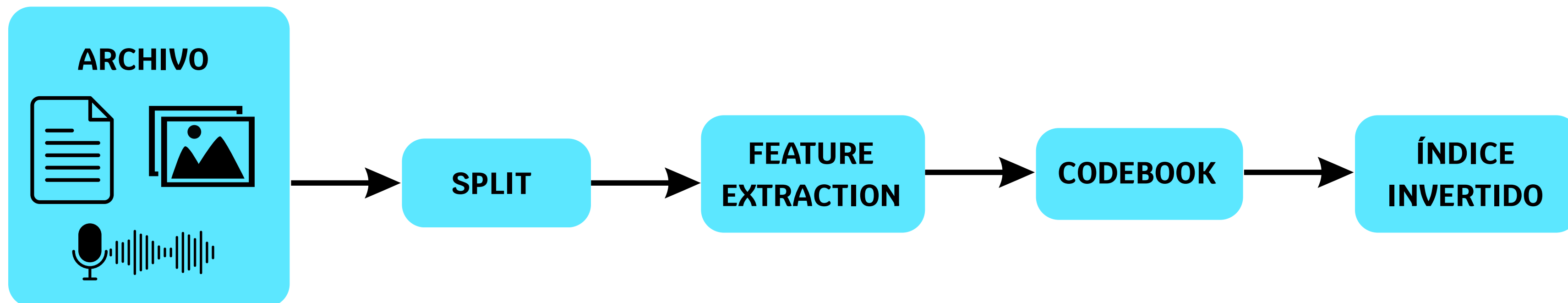


TECNICAS INDEXACION

Índice	Estructura	build	search	insert	delete	El planner lo elige para
B+Tree	árbol balanceado, hojas enlazadas	$O(n \log n)$	$O(\log n)$, rango $O(\log n + k)$	$O(\log n)$	$O(\log n)$	predicados RANGE / BETWEEN
ISAM	páginas primarias estáticas + overflow chains	$O(n \log n)$	$O(\log n) +$ cadena	$O(1)$ a overflow	$O(\text{páginas candidatas})$	alternativa estática por DDL
Hash extendible	directorio + buckets con split	$O(n)$	$O(1)$ esperado	$O(1)$ amortizado	$O(1)$	predicados de igualdad
R-Tree	árbol espacial (librería <code>rtree</code>)	$O(n \log n)$	knn y rango espacial $O(\log n)$ esperado	$O(\log n)$	por intersección	WITHIN espacial
Inverted (SPIMI)	postings por término, TF-IDF coseno	bloques + merge externo	$O(\text{términos de la query por sus postings})$	$O(\text{términos del doc})$	$O(\text{vocabulario})$	búsqueda de texto
MultimediaKNN	histogramas + inverted lists por visual word	$O(n \text{ por words activas})$	$O(\text{candidatos por dim})$	$O(\text{words activas})$	$O(\text{words activas})$	predicados KNN



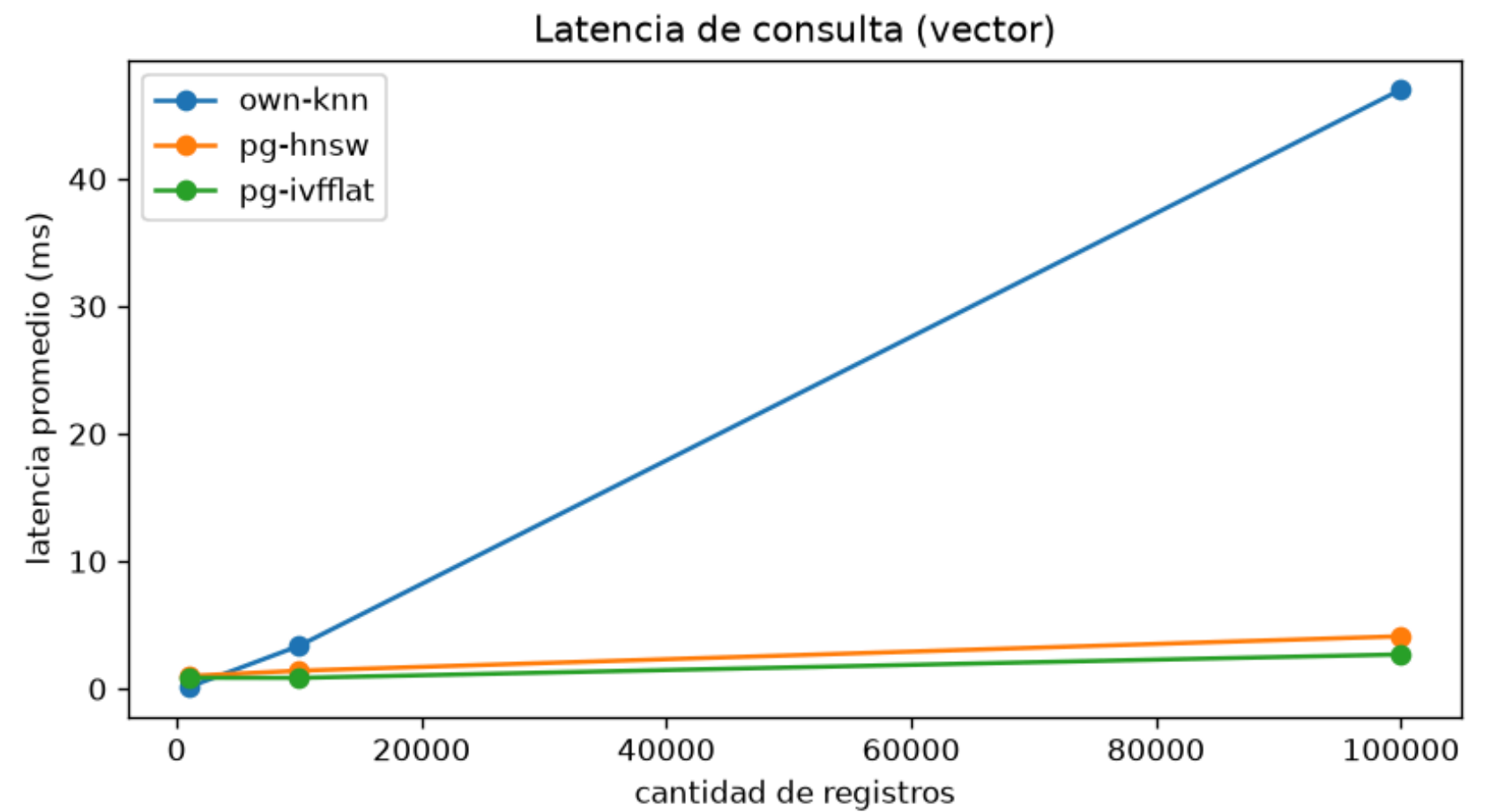
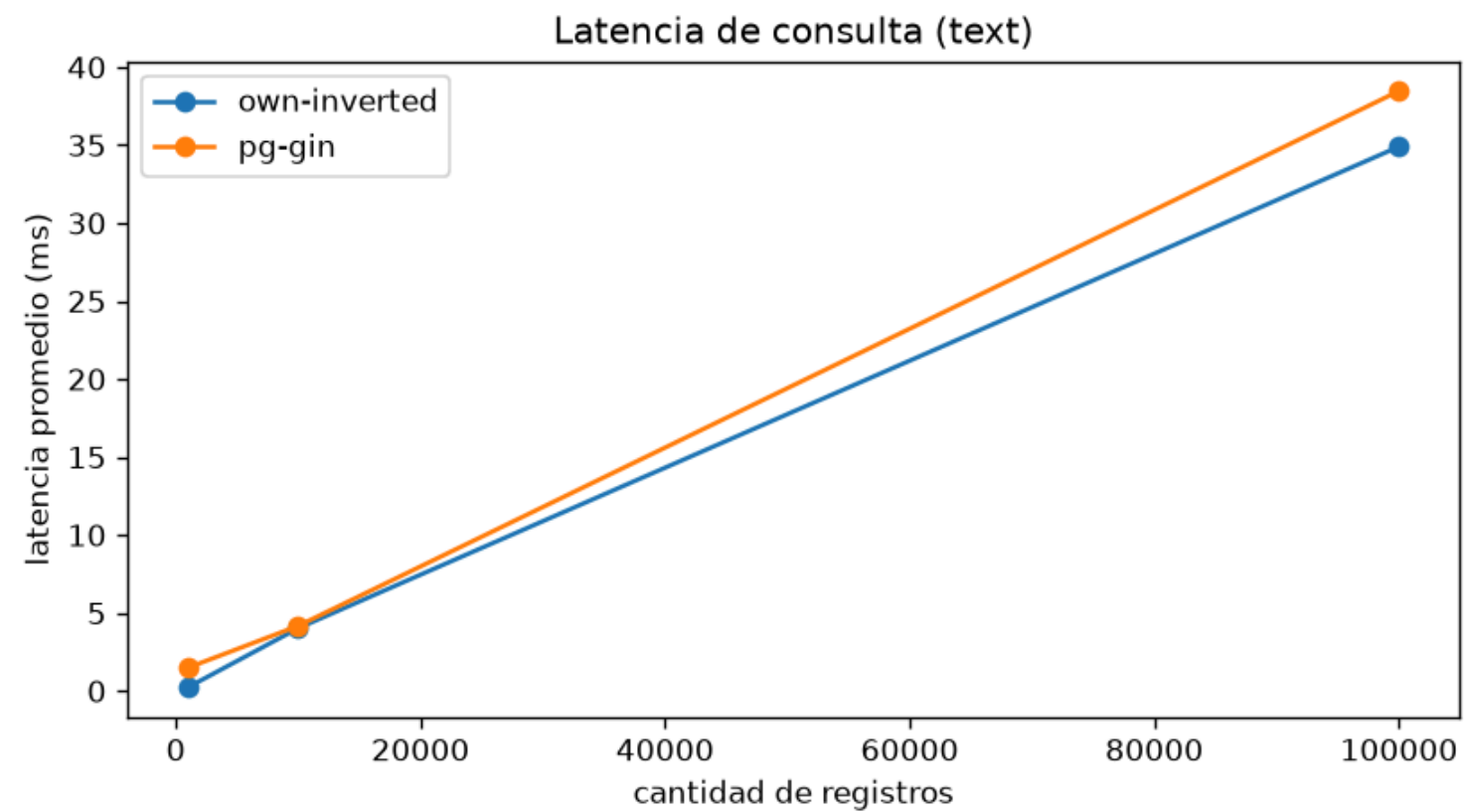
PIPELINE MULTIMODAL



MÉTRICAS

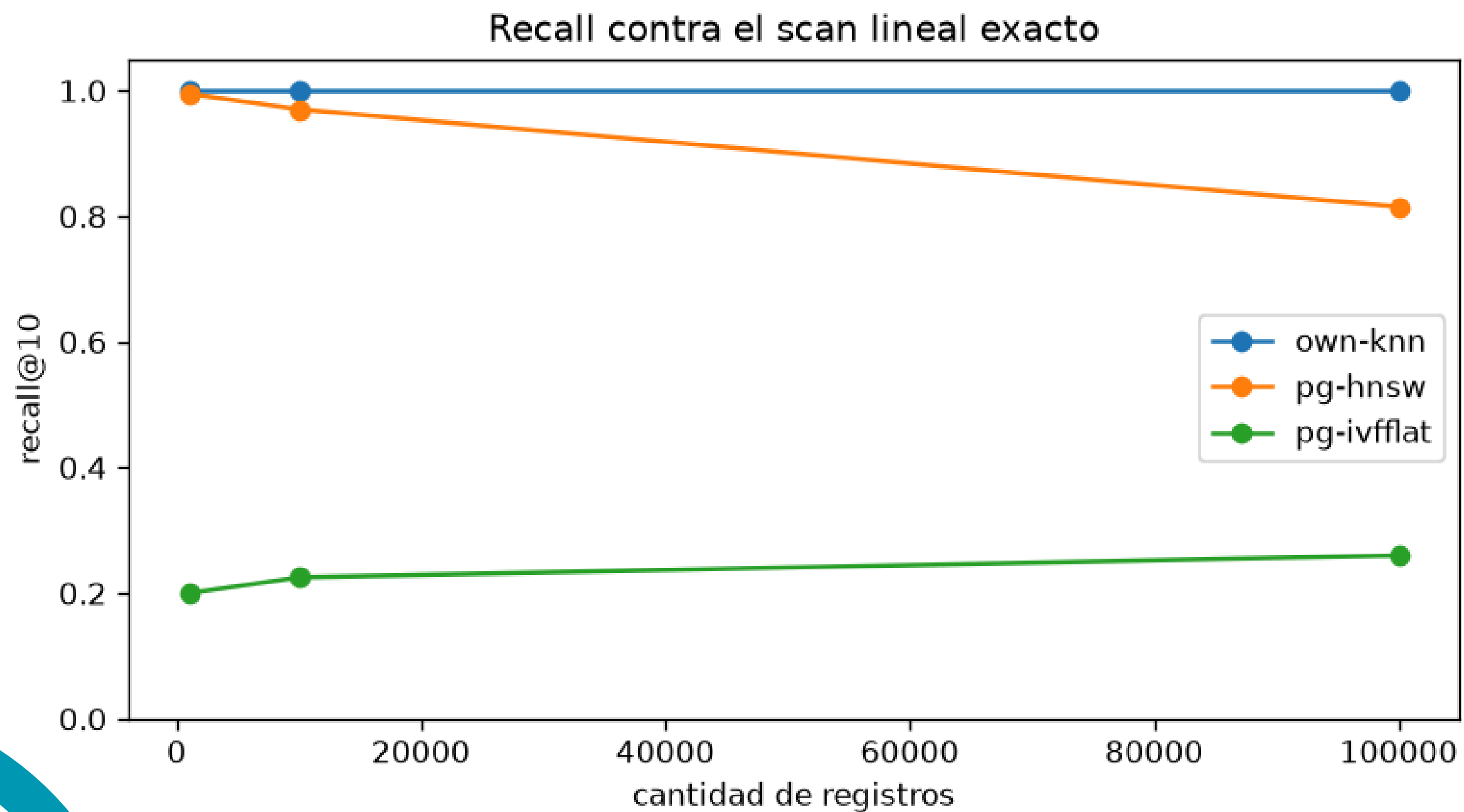
- **Latencia:** tiempo de pared por consulta
- **Throughput:** consultas por segundo
- **Recall@k:** fracción del top-k exhaustivo que el índice recupera
- **Overlap textual:** similitud de Jaccard promedio entre el top-k del inverted index propio y el top-k de GIN con ts_rank para las mismas consultas (columna overlap_at_k).
- **Memoria:** dos mediciones además de la cota estructural
- **I/O:** disk_reads, disk_writes y pages_allocated

RESULTADOS-LATENCIA



RESULTADOS

Comparación contra PostgreSQL



CONCLUSIONES

- El paradigma unificado (extractor, codebook, histogramas, inverted index) funciona de punta a punta para texto e imágenes, con contratos que permiten agregar una modalidad nueva sin tocar el executor.
- Centralizar el I/O en el BufferManager hizo trivial medir el costo real por consulta: el executor solo calcula el delta de contadores del storage y la métrica llega intacta hasta el panel del frontend.
- SPIMI con bloques en memoria secundaria y merge k-way streaming escala más allá de la RAM del proceso de build, cumpliendo la forma canónica del algoritmo.
- La comparación contra PostgreSQL es reproducible y honesta: en texto el engine propio gana con N chico y empata con GIN en 100K pagando mucha más memoria, y en vectores gana recall (1.000 contra el scan exacto) pero pierde latencia contra IVFFlat y HNSW al crecer N.

GRACIAS